

ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เอกสารประกอบการสอน

040223101 การเขียนโปรแกรมคอมพิวเตอร์สำหรับคณิตศาสตร์ 1

(Computer Programming for Mathematics 1)

บทที่ 8

ดิกชันนารีและเซต

(Dictionaries and Sets)

ผู้สอน: รศ.ดร.อนุชิต จิตพัฒนกุล

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อศึกษาบทนี้จบ นักศึกษาสามารถ

- สร้างและเข้าถึงข้อมูลในดิกชันนารีผ่านคู่คีย์-ค่า (key-value) ได้
- ใช้เมทอดของดิกชันนารี เช่น keys, values, items, get ได้
- สร้างเซตและดำเนินการเซต ได้แก่ ยูเนียน อินเตอร์เซกชัน และผลต่างได้
- เลือกใช้ดิกชันนารีและเซตให้เหมาะสมกับลักษณะของปัญหาได้
- ประยุกต์ดิกชันนารีและเซตในการนับและจัดกลุ่มข้อมูลได้

CLO: เนื้อหาบทนี้สอดคล้องกับผลลัพธ์การเรียนรู้ของรายวิชา (CLO 4)

บทที่ 8 ดิกชันนารีและเซต

ลิสต์เก็บข้อมูลเป็นลำดับและเข้าถึงด้วยดัชนีที่เป็นตัวเลข แต่บางครั้งเราอยากเข้าถึงข้อมูลด้วย "ชื่อ" เช่น ค้นคะแนนจากชื่อ

นักศึกษา **ดิกชันนารี (dictionary)** ตอบโจทย์นี้ด้วยการเก็บข้อมูลเป็นคู่คีย์-ค่า ส่วน **เซต (set)** เก็บข้อมูลที่ไม่ซ้ำกันและรองรับการดำเนินการทางเซต บทนี้จะศึกษาทั้งสองโครงสร้าง

8.1 ดิกชันนารีคืออะไร

ดิกชันนารีเก็บข้อมูลเป็นคู่ **คีย์ (key)** : **ค่า (value)** สร้างด้วยวงเล็บปีกกา { } การเข้าถึงค่าใช้คีย์ แทนดัชนีตัวเลข คีย์ต้องไม่ซ้ำกันและเป็นข้อมูลที่ไม่เปลี่ยนแปลงไม่ได้ เช่น ข้อความหรือตัวเลข

ไฟล์ 08dict1.py

```
student = {"name": "Somchai", "age": 19, "gpa": 3.5}
print(student["name"])
print(student["gpa"])
student["age"] = 20      # แก้ไขค่า
student["major"] = "Math" # เพิ่มคู่ใหม่
print(student)
```

ผลลัพธ์:

```
Somchai
3.5
{'name': 'Somchai', 'age': 20, 'gpa': 3.5, 'major': 'Math'}
```

ข้อควรระวัง: หากเข้าถึงคีย์ที่ไม่มีอยู่ด้วย student["xyz"] จะเกิด KeyError ควรใช้เมทอด get() ที่คืนค่า None (หรือค่าปริยายที่กำหนด) แทนการเกิดข้อผิดพลาด

8.2 เมทอดของดิกชันนารี

เมทอด	ความหมาย
d.keys()	คืนคีย์ทั้งหมด
d.values()	คืนค่าทั้งหมด
d.items()	คืนคู่ (คีย์, ค่า) ทั้งหมด
d.get(k, default)	คืนค่าของคีย์ k ถ้าไม่มีคืน default
d.pop(k)	ลบและคืนค่าของคีย์ k
k in d	ตรวจว่ามีคีย์ k หรือไม่ (True/False)

เราวนซ้ำดิกชันนารีด้วยลูป for ได้ทั้งทางคีย์และทางคีย์-ค่า

ไฟล์ 08dict2.py

```
scores = {"Ann": 85, "Bob": 90, "Cat": 78}
for name, score in scores.items():
    print(f"{name} ได้ {score} คะแนน")
print("ค่าเฉลี่ย:", sum(scores.values()) / len(scores))
```

ผลลัพธ์:

```
Ann ได้ 85 คะแนน
Bob ได้ 90 คะแนน
Cat ได้ 78 คะแนน
ค่าเฉลี่ย: 84.33333333333333
```

8.3 ตัวอย่างประยุกต์: นับความถี่

ดิกชันนารีเหมาะมากกับการนับความถี่ของข้อมูล ตัวอย่างนับจำนวนครั้งที่แต่ละตัวอักษรปรากฏ

ไฟล์ 08freq.py

```
text = "mathematics"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

ผลลัพธ์:

```
{ 'm': 2, 'a': 2, 't': 2, 'h': 1, 'e': 1, 'i': 1, 'c': 1, 's': 1 }
```

8.4 เซตคืออะไร

เซต (set) คือกลุ่มของสมาชิกที่ **ไม่ซ้ำกัน** และไม่มีลำดับ สร้างด้วยวงเล็บปีกกา { } หรือฟังก์ชัน set() เมื่อใส่ค่าข้างลงไป เซตจะเก็บไว้เพียงค่าเดียว จึงเหมาะกับการกำจัดค่าซ้ำ

ไฟล์ 08set1.py

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique = set(nums)
print(unique)
print("จำนวนค่าที่ไม่ซ้ำ:", len(unique))
unique.add(5)
unique.discard(1)
print(unique)
```

ผลลัพธ์:

```
{1, 2, 3, 4}
จำนวนค่าที่ไม่ซ้ำ: 4
{2, 3, 4, 5}
```

8.5 การดำเนินการเซต

เซตรองรับการดำเนินการทางคณิตศาสตร์เช่นเดียวกับเซตในวิชาคณิตศาสตร์

การดำเนินการ	ตัวดำเนินการ	เมทอด	ความหมาย
ยูเนียน	$A \mid B$	<code>A.union(B)</code>	สมาชิกที่อยู่ใน A หรือ B
อินเตอร์เซกชัน	$A \& B$	<code>A.intersection(B)</code>	สมาชิกที่อยู่ทั้ง A และ B
ผลต่าง	$A - B$	<code>A.difference(B)</code>	สมาชิกที่อยู่ใน A แต่ไม่อยู่ใน B
ผลต่างสมมาตร	$A \wedge B$	<code>A.symmetric_difference(B)</code>	สมาชิกที่อยู่ใน A หรือ B แต่ไม่ทั้งคู่

ไฟล์ 08setop.py

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7}
print("ยูเนียน:", A | B)
print("อินเตอร์เซกชัน:", A & B)
print("ผลต่าง A-B:", A - B)
print("ผลต่างสมมาตร:", A ^ B)
```

ผลลัพธ์:

```
ยูเนียน: {1, 2, 3, 4, 5, 6, 7}
อินเตอร์เซกชัน: {4, 5}
ผลต่าง A-B: {1, 2, 3}
ผลต่างสมมาตร: {1, 2, 3, 6, 7}
```

8.6 สรุปท้ายบท

ดิกชันนารีเก็บข้อมูลเป็นคู่คีย์-ค่า เข้าถึงด้วยคีย์แทนดัชนีตัวเลข เหมาะกับการจับคู่ข้อมูลและการนับ ความถี่ เมทอดสำคัญคือ

keys, values, items และ get เซตเก็บสมาชิกที่ไม่ซ้ำกันและไม่มีลำดับ เหมาะกับการกำจัดค่าซ้ำและการดำเนินการทางเซต (ยูเนียน อินเตอร์เซกชัน ผลต่าง) ทั้งสองโครงสร้างเสริม ลิสต์ให้เราเลือกใช้ได้เหมาะสมกับลักษณะของปัญหา

แบบฝึกหัดท้ายบท

- จงสร้างดิกชันนารีเก็บชื่อและอายุของเพื่อน 3 คน แล้วแสดงชื่อของคนที่มีอายุมากที่สุด
- จงเขียนโปรแกรมนับความถี่ของคำในประโยคที่รับเข้ามา (แยกคำด้วยช่องว่าง)
- จงเขียนโปรแกรมหาจำนวนค่าที่ไม่ซ้ำกันในลิสต์ [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
- กำหนดเซต $A = \{2,4,6,8,10\}$ และ $B = \{3,6,9\}$ จงแสดงผลของยูเนียน อินเตอร์เซกชัน และ $A-B$

เฉลยแบบฝึกหัดบทที่ 8

ข้อ 2 นับความถี่ของคำ

```
sentence = input("ใส่ประโยค: ")
words = sentence.split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq)
```

ผลลัพธ์:

```
ใส่ประโยค: a b a c b a
{'a': 3, 'b': 2, 'c': 1}
```

ข้อ 3 จำนวนค่าที่ไม่ซ้ำ

```
nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
print("ค่าที่ไม่ซ้ำ:", set(nums))
print("จำนวน:", len(set(nums)))
```

ผลลัพธ์:

```
ค่าที่ไม่ซ้ำ: {1, 2, 3, 4, 5, 6, 9}
จำนวน: 7
```

ข้อ 4 การดำเนินการเซต

```
A = {2, 4, 6, 8, 10}
B = {3, 6, 9}
print(A | B)
print(A & B)
print(A - B)
```

ผลลัพธ์:

```
{2, 3, 4, 6, 8, 9, 10}
{6}
{2, 4, 8, 10}
```